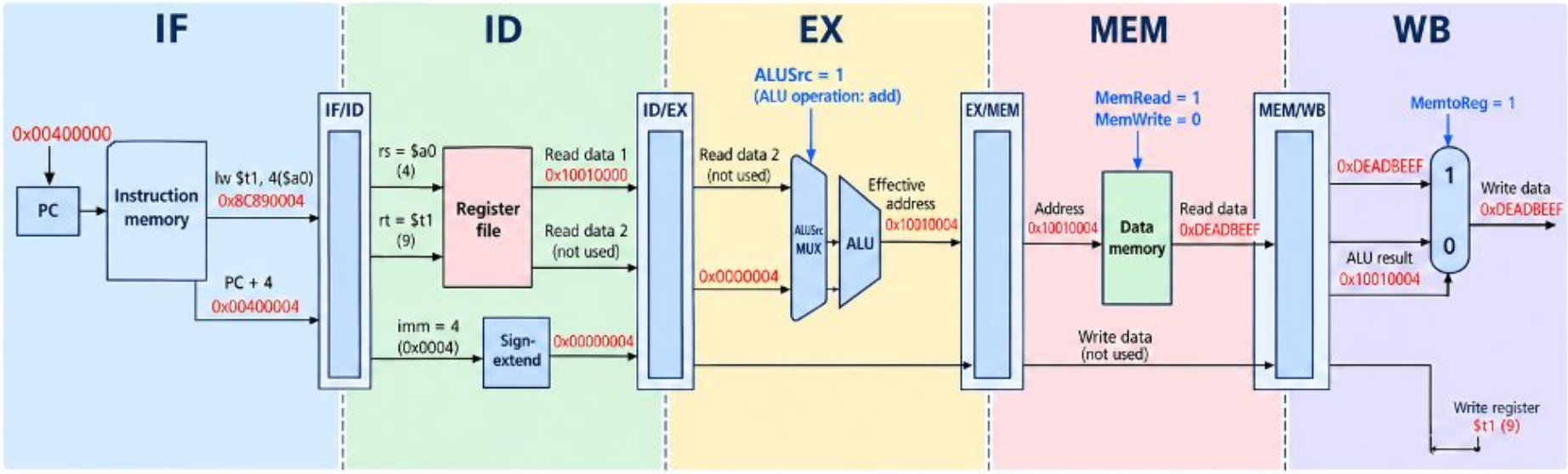
# Chapter 4

## Pipelining

# Pipeline Operation (Recap)

- Cycle-by-cycle flow of instructions through the pipelined datapath
  - "Single-clock-cycle" pipeline diagram
    - Shows pipeline usage in a single cycle
    - Highlight resources used
  - c.f. "multi-clock-cycle" diagram
    - Graph of operation over time
- We'll look at "single-clock-cycle" diagrams for load & store

# Complete Load Inst. Tracing




### IF — Instruction Fetch

**Instruction:**
lw $t1, 4($a0)

**PC:**
0x00400000

Instruction immory →
0x8C890004

**PC + 4:**
0x00400004

**Outputs to IF/ID:**
- Instruction = lw $t1, 4($a0)
- PC + 4 = 0x00400004

### ID — Decode & Register Read

**Register file read:**
rs = $a0 → 0x10010000
rt = $t1 → (not used)

**Sign-extend immediate:**
4 → 0x00000004

**Outputs to ID/EX:**
- Read data 1 = 0x10010000
- Read data 2 = (not used)
- Imm = 0x00000004
- rs = 4, rt = 9
- Control signals

### EX — Address Calculation

ALUSrc = 1 → use immediate

**ALU operation: add**

0x10010000 + 0x00000004
= 0x10010004

**Outputs to EX/MEM:**
- ALU result = 0x10010004
- Read data 2 = (not used)
- rt = 9 (destination register)
- Control signals

### MEM — Memory Read

MemRead = 1, MemWrite = 0

**Address:**
0x10010004

**Data memory read:**
0xDEADBEEF

**Outputs to MEM/WB:**
- Read data = 0xDEADBEEF
- ALU result = 0x10010004
- rt = 9
- Control signals

### WB — Write Back

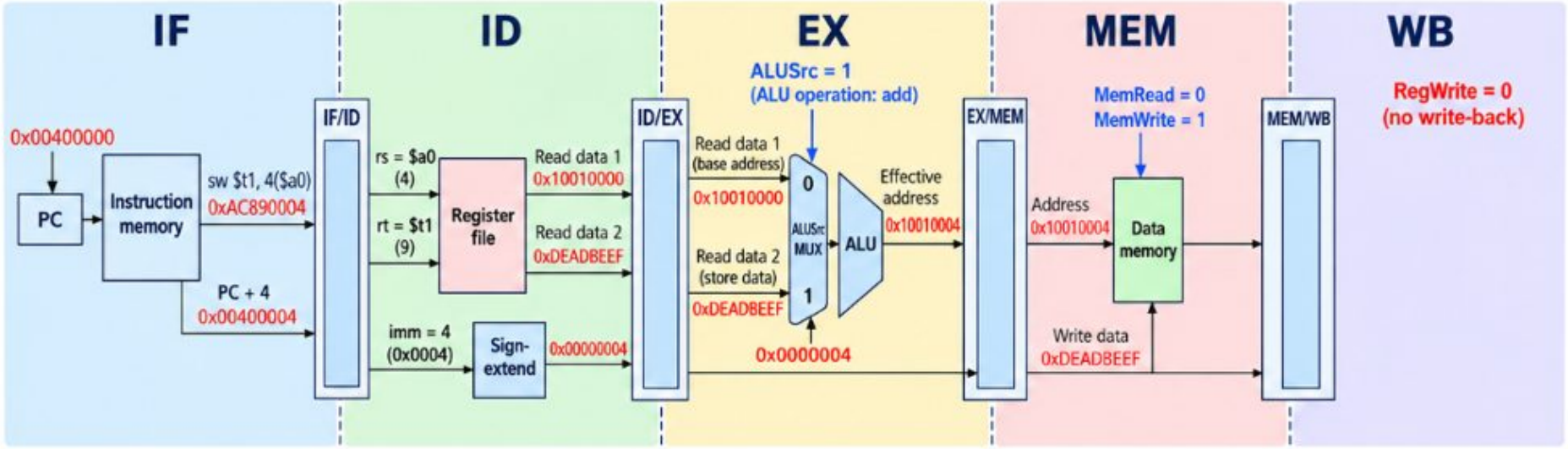
MemtoReg = 1

Write register = $t1 (9)

Write data = 0xDEADBEEF

**Result:**
$t1 = 0xDADBEEF

# Complete Store Inst. Tracing




## IF — Instruction Fetch

**Instruction:**
sw $t1, 4($a0)

**PC:**
0x00400000

**Instruction memory →**
0xAC890004

**PC + 4:**
0x00400004

**Outputs to IF/ID:**
- Instruction = sw $t1, 4($a0)
- PC + 4 = 0x00400004

## ID — Decode & Register Read

**Register file read:**
rs = $a0 → 0x10010000
rt = $t1 → 0xDEADBEEF

**Sign-extended immediate:**
4 → 0x00000004

**Outputs to ID/EX:**
- Read data 1 = 0x10010000
- Read data 2 = 0xDEADBEEF
- Imm = 0x00000004
- rs = 4, rt = 9
- Control signals

## EX — Address Calculation

ALUSrc = 1 → use immediate

**ALU operation:** add
0x10010000 + 0x00000004
= 0x10010004

**Outputs to EX/MEM:**
- ALU result = 0x10010004
- Read data 2 = 0xDEADBEEF (store data)
- rt = 9 (for completeness)
- Control signals

## MEM — Memory Write

MemRead = 0, MemWrite = 1

**Address:**
0x10010004

**Write data:**
0xDEADBEEF

**Memory operation:**
Mem[0x10010004] ← 0xDEADBEEF

**Outputs to MEM/WB:**
- ALU result = 0x10010004 (not used)
- Control signals

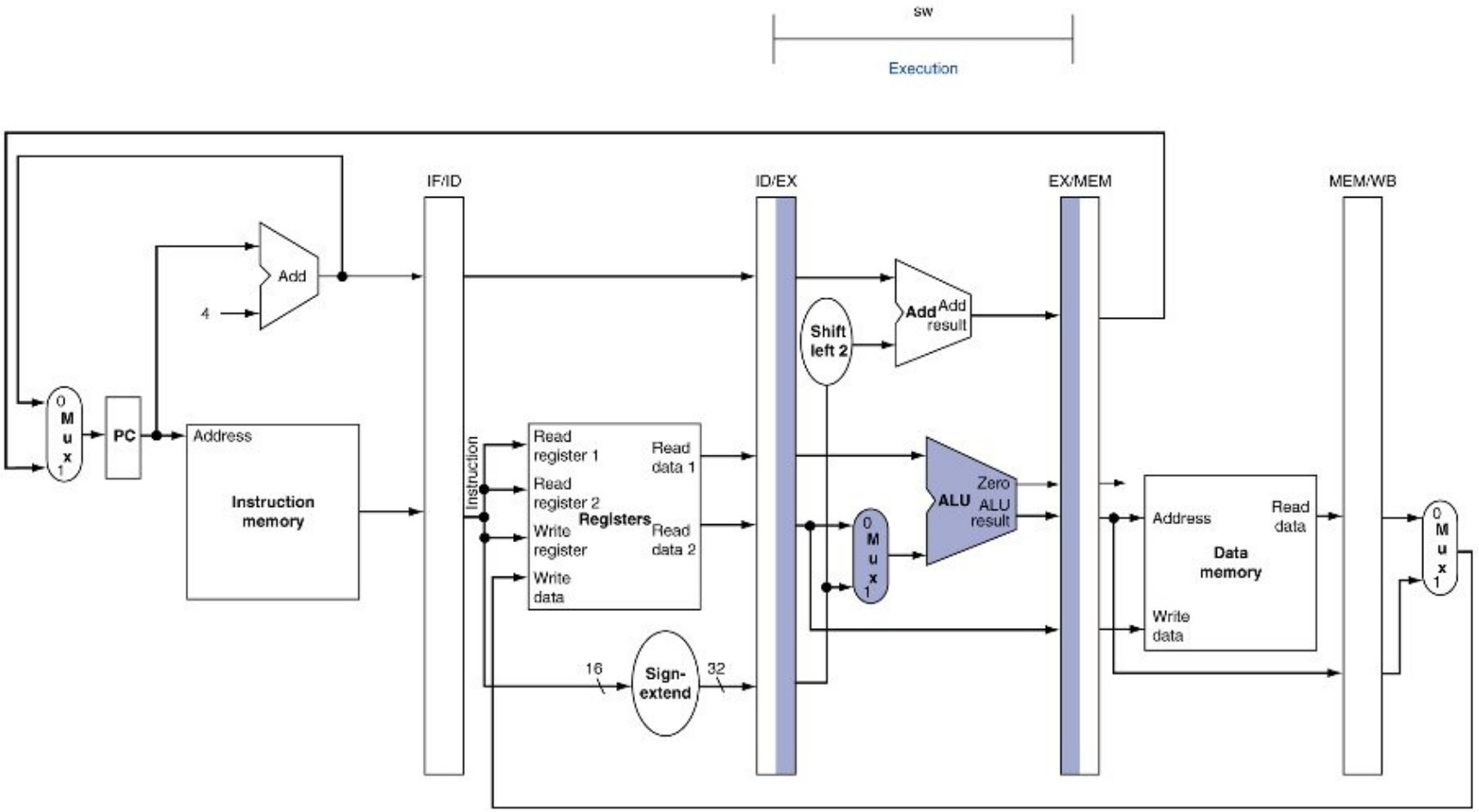
## WB — Write Back

RegWrite = 0
(no register write)

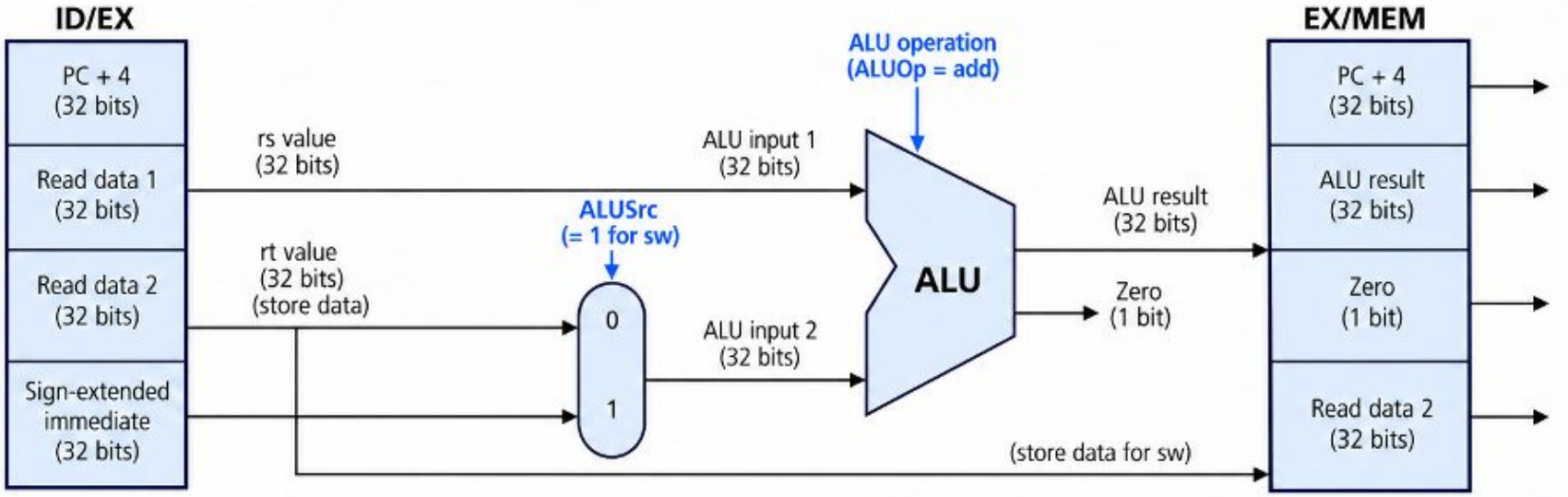
**Result:**
No change to register file.

*(Memory updated in MEM stage)*

# EX for Store


sw
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
16
Sign-extend
32

# Execute for Store (Cont.)




| Operation | Load (lw) | Store (sw) | R-type (add, sub, ...) |
|---|---|---|---|
| **ALU operation** | Add (address calc.) | Add (address calc.) | According to funct (add, sub, and, ...) |
| **ALU input 1** | rs value (base address) | rs value (base address) | rs value |
| **ALU input 2 (ALUSrc)** | sign-extended imm (offset) (ALUSrc = 1) | sign-extended imm (offset) (ALUSrc = 1) | rt value (ALUSrc = 0) |
| **ALU result** | effective address | effective address | arithmetic/logic result |
| **Zero output** | not used | not used | set if result = 0 (used by branches) |
| **Forward to EX/MEM** | ALU result (effective address) | ALU result (effective address), Read data 2 (store data) | ALU result |

## Example: sw $t1, 4($a0)

**Inputs from ID/EX**

| | |
|---|---|
| Read data 1 (rs = $a0) | 0x10010000 |
| Read data 2 (rt = $t1) | 0xDEADBEEF |
| Sign-extended immediate | 0x00000004 |

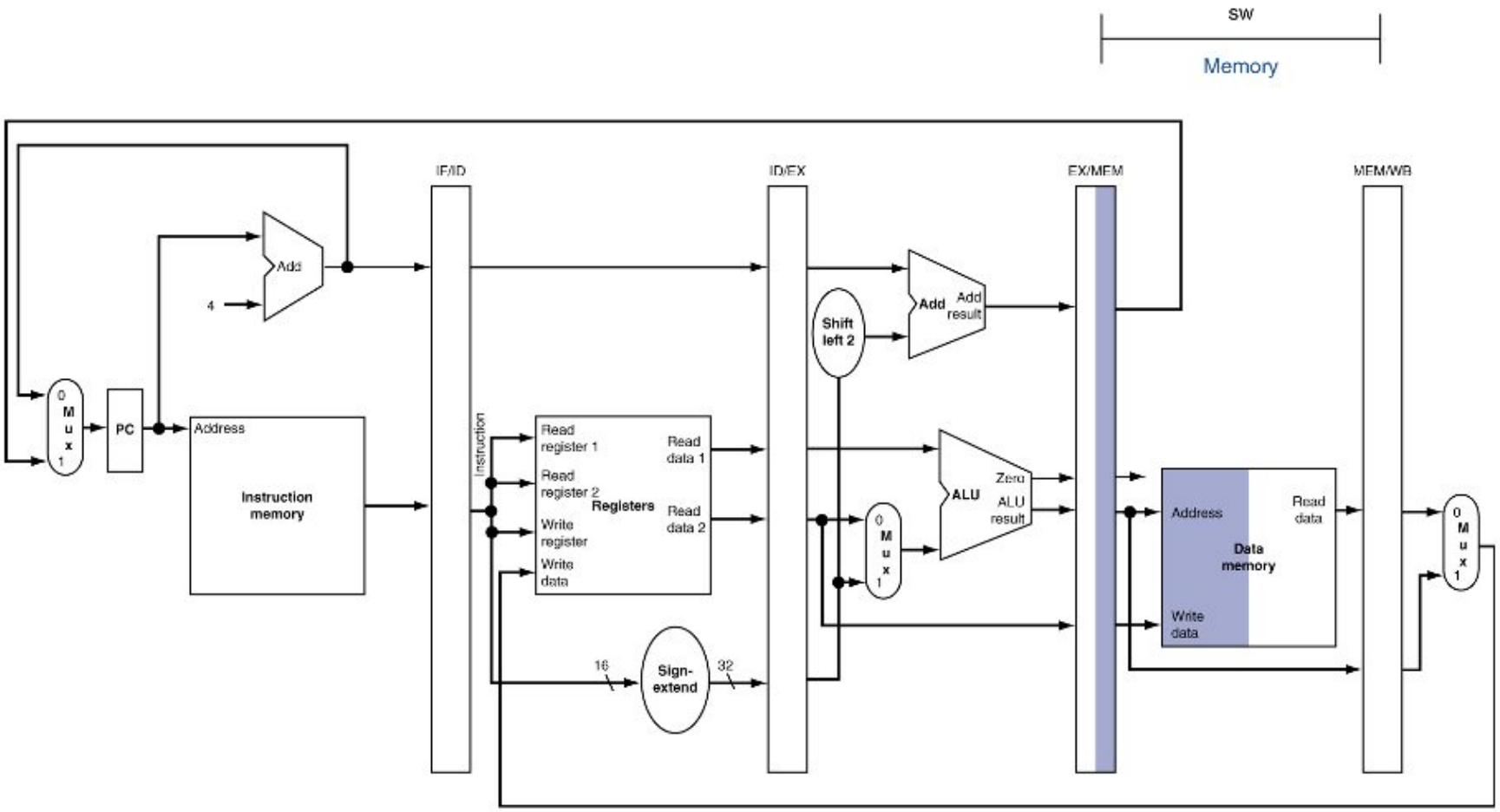
**ALU operation in EX stage**

ALU result = 0x10010000 + 0x00000004 = **0x10010004**

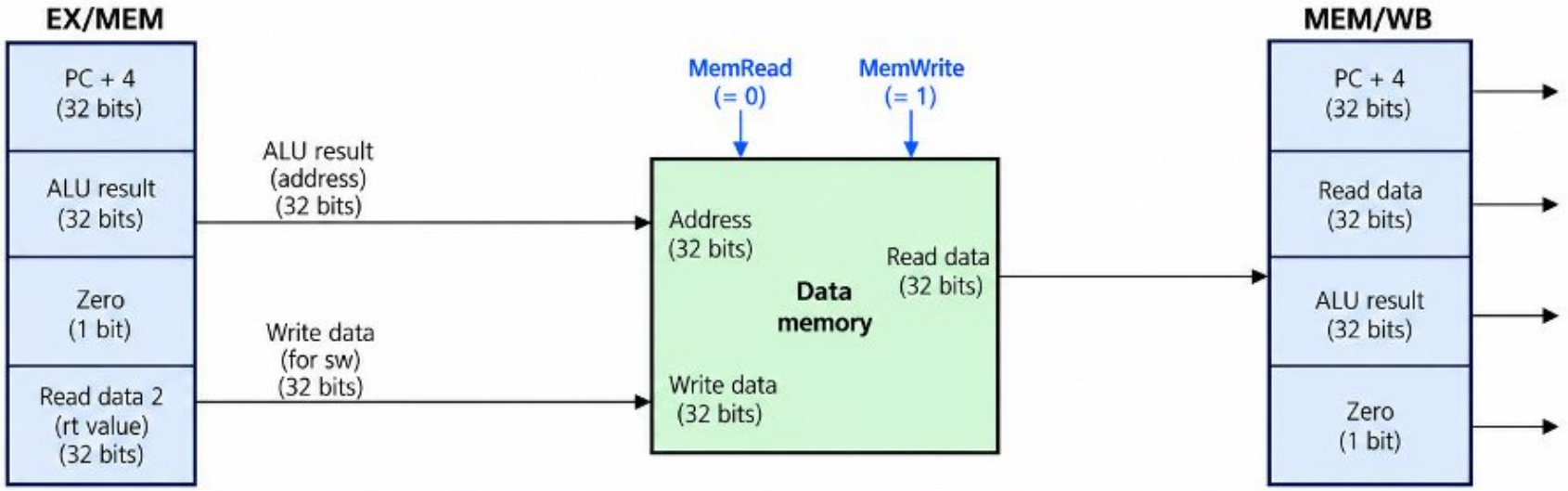
**Outputs to EX/MEM**

| | |
|---|---|
| ALU result | 0x10010004 (effective address) |
| Zero | not used (—) |
| Read data 2 | 0xDEADBEEF (store data for sw) |

# MEM for Store



# (Cont.)




| Operation | Load (lw) | Store (sw) | R-type (add, sub, ...) |
|---|---|---|---|
| **Memory access** | Read from Data memory | Write to Data memory | No memory access |
| **Address input** | ALU result (effective address) | ALU result (effective address) | ALU result (usually not used) |
| **Write data** | — | rt value (from EX/MEM) | — |
| **Read data** | value from Data memory | — | — |
| **Control signal** | MemRead = 1<br>MemWrite = 0 | **MemRead = 0**<br>**MemWrite = 1** | MemRead = 0<br>MemWrite = 0 |
| **Outputs to MEM/WB** | Read data (from memory) | ALU result (effective address) | ALU result (ALU computation) |
| **Zero output** | not used | not used | not used |

## Example: sw $t1, 4($a0)

**Inputs from EX/MEM**

| | |
|---|---|
| ALU result (address) | 0x10010004 |
| Write data (rt = $t1) | 0xDEADBEEF |
| MemRead | 0 |
| MemWrite | 1 |

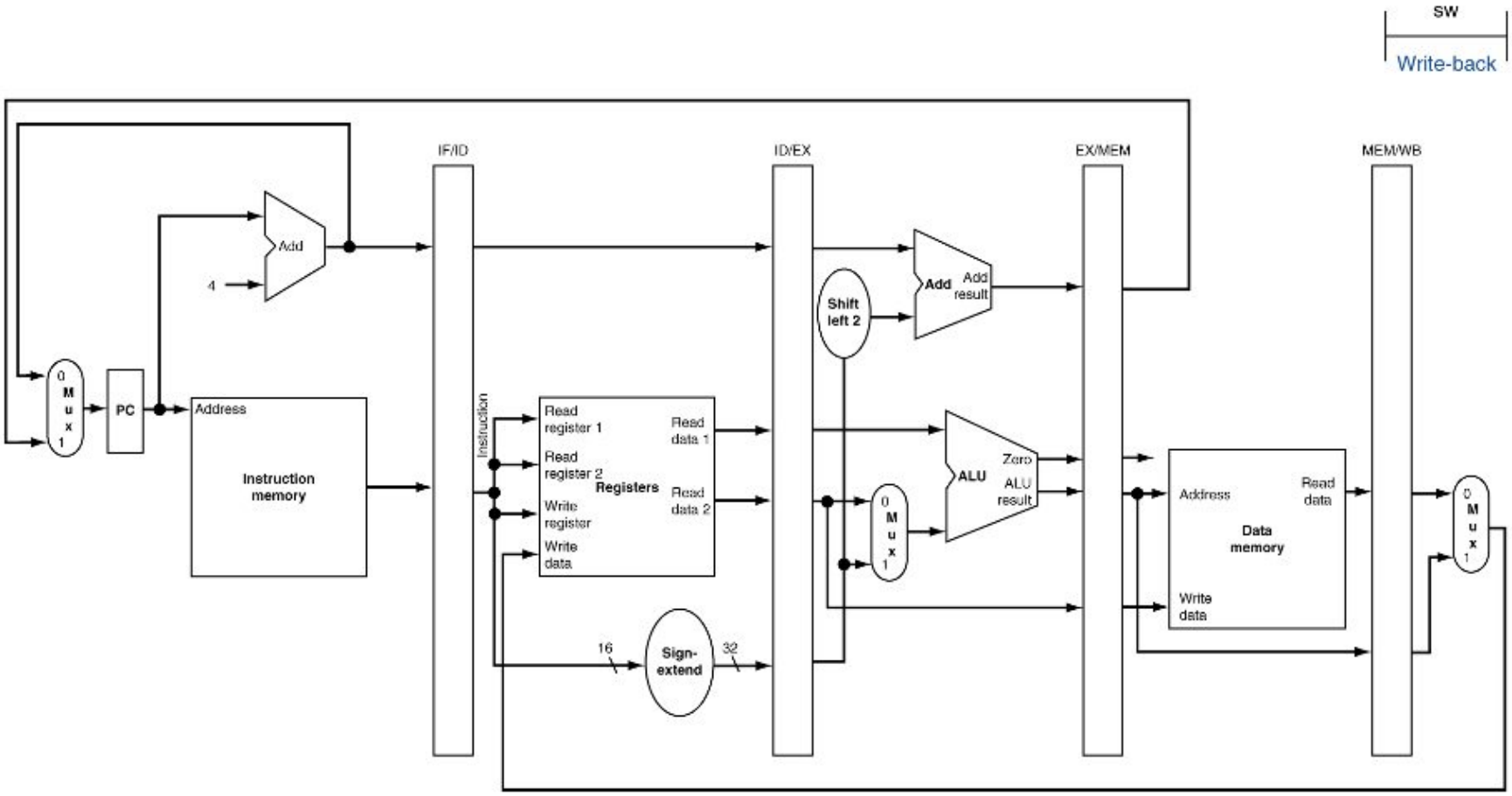
**Memory access in MEM stage**

**Data memory write:**
Mem[0x10010004] ← **0xDEADBEEF**

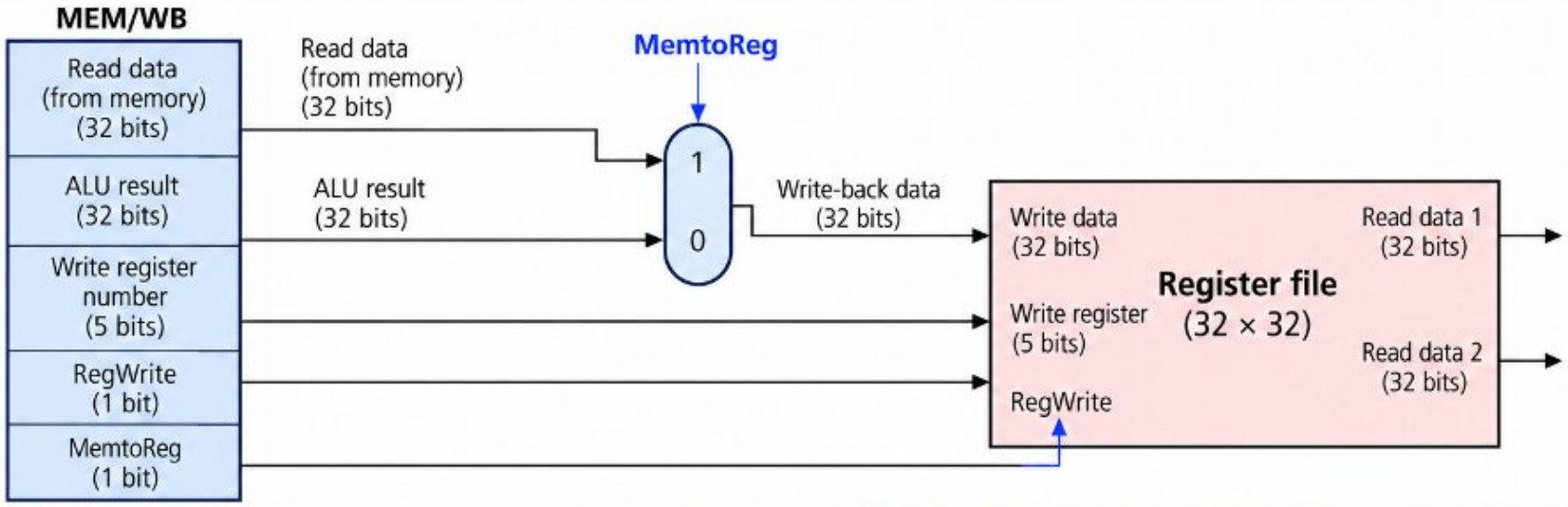
**Outputs to MEM/WB**

| | |
|---|---|
| PC + 4 | PC + 4 (sequential address) |
| Read data | not used (—) |
| ALU result | 0x10010004 (effective address) |
| Zero | not used (—) |

# WB for Store


SW
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M
u
x
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign-extend
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data

# Write Back Store (Cont.)




| Operation | Load (lw) | Store (sw) | R-type (add, sub, ...) |
|---|---|---|---|
| **Data written to register file** | Data from memory | None | ALU result |
| **MemtoReg** | 1 | (don't care) | 0 |
| **RegWrite** | 1 | 0 | 1 |
| **Write register** | rt (from MEM/WB) | — | rd (from MEM/WB) |
| **Write-back data** | Read data (from memory) | — | ALU result |
| **Inputs from MEM/WB** | Read data ALU result Write register MemtoReg, RegWrite | (not used for write) Write register MemtoReg, RegWrite | Read data ALU result Write register MemtoReg, RegWrite |

## Example: sw $t1, 4($a0)

**Inputs from MEM/WB**

| | |
|---|---|
| Read data (from memory) | not used (—) |
| ALU result (address) | 0x10010004 |
| Write register number (rt = $t1) | 9 (5 bits) |
| MemtoReg | don't care (X) |
| RegWrite | 0 |

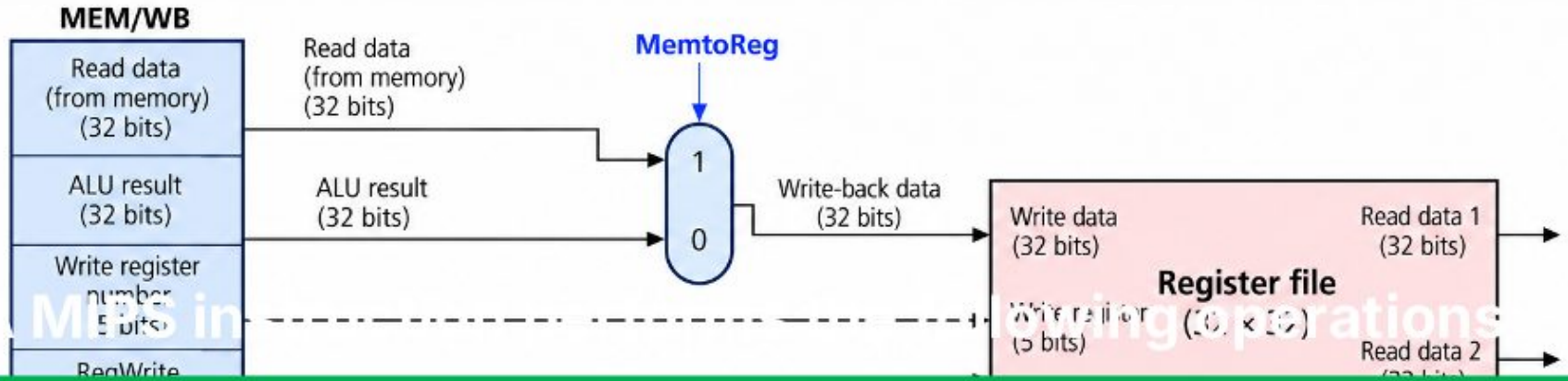
**Write-back operation**

RegWrite = 0 → no register write

**Result in register file**

| | |
|---|---|
| Register | $t1 (register number 9) |
| Written value | No change (—) |

# Write Back Store (Cont.)




In EX, the ALU calculates 0x10010000 + 0x00000008.

In MEM, the value 0x12345678 is written to the calculated address. In WB, no register is updated.

Which instruction is being executed?

A1.

sw $t1, 8($a0)

Assuming $a0 = 0x10010000 and $t1 = 0x12345678.

The effective address is 0x10010008, and the instruction

# Load vs. Store

| Stage | Load (lw) Inputs | Store (sw) Inputs |
|---|---|---|
| IF | PC | PC |
| ID | Instruction, PC + 4 | Instruction, PC + 4 |
| EX | Base address (rs), immediate | Base address (rs), immediate, store data (rt) |
| MEM | Effective address | Effective address, store data |
| WB | Memory data, destination register (rt) | None required |

| Stage | Load (lw) Outputs | Store (sw) Outputs |
|---|---|---|
| IF | Instruction, PC + 4 | Instruction, PC + 4 |
| ID | Base address, immediate, destination register (rt) | Base address, store data (rt), immediate |
| EX | Effective address | Effective address, store data |
| MEM | Memory read data | Memory updated |
| WB | Register updated | No register write |

# Load vs. Store

| Stage | Load (lw) Inputs | Store (sw) Inputs |
|---|---|---|
| IF | PC | PC |
| ID | Instruction, PC + 4 | Instruction, PC + 4 |
| EX | Base address (rs), | Base address (rs), immediate, |
| WB | Register updated | No register write |

2. Consider the following instruction: sw $t2, 12($a0)

Assume $a0 = 0x10010000 and $t2 = 0xABCDEF12.

What values are stored in the EX/MEM pipeline register for the ALU result and Read data 2? What happens in the MEM stage?

A2.

ALU result: 0x1001000C

Read data 2: 0xABCDEF12

MEM stage: Mem[0x1001000C] ← 0xABCDEF12

Control signals: MemRead = 0, MemWrite = 1

# Overview

Time (in clock cycles) →

| Instruction | CC 1 | CC 2 | CC 3 | CC 4 | CC 5 | CC 6 | CC 7 |
|---|---|---|---|---|---|---|---|
| lw $t1, 4($a0) | IF | ID | EX | MEM | WB | | |
| sw $t2, 8($a0) | | IF | ID | EX | MEM | — | |
| add $t3, $t4, $t5 | | | IF | ID | EX | MEM | WB |

## Instruction Overlapping

- Multiple instructions execute in different pipeline stages during the same clock cycle.
- In each clock cycle, different stages work on different instructions.

## Pipeline Latency

- An instruction passes through five pipeline stages:

IF | ID | EX | MEM | WB

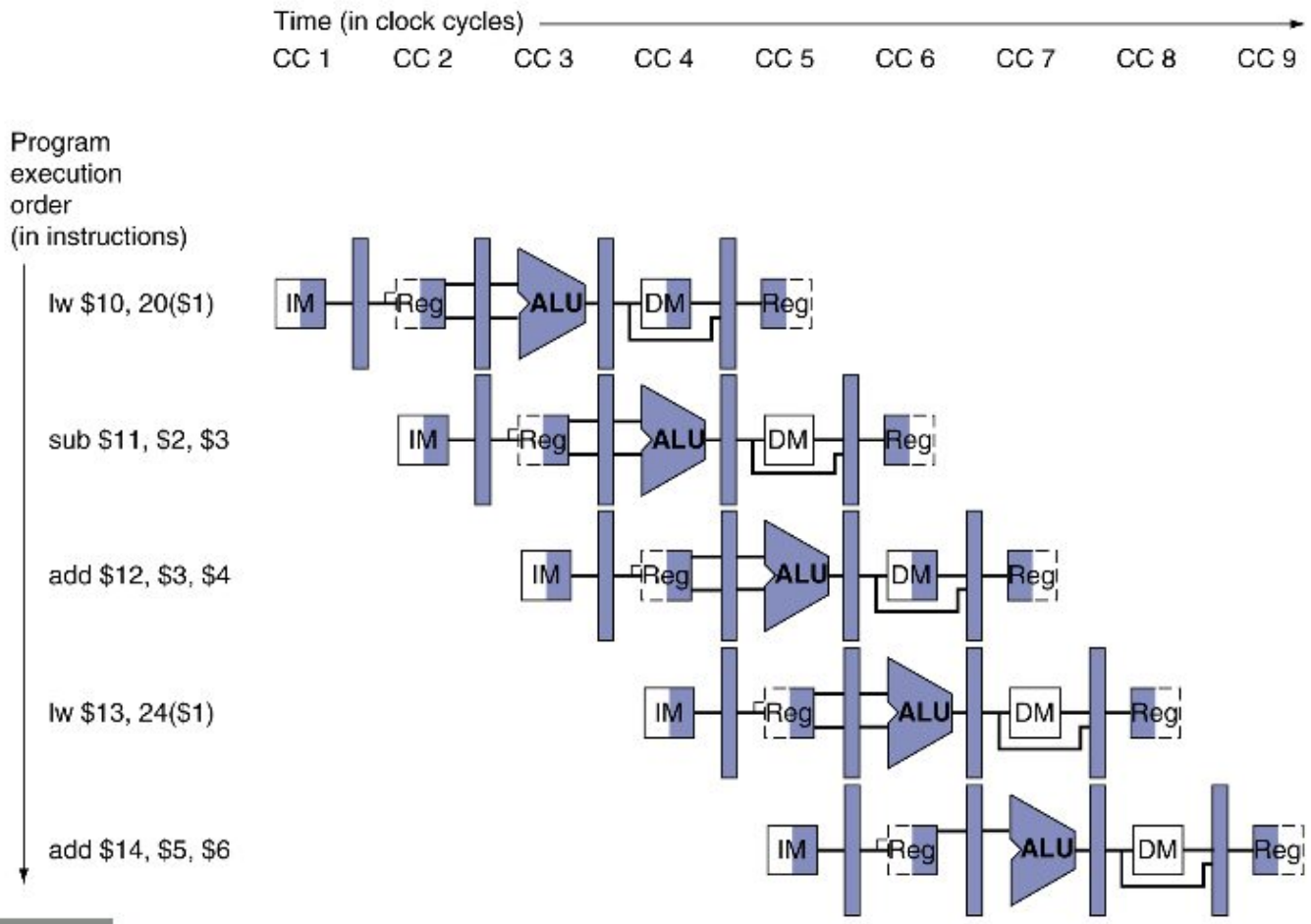
- It takes 5 clock cycles for an instruction to complete.

## Pipeline Throughput

- After the pipeline is filled, ideally one instruction can complete per clock cycle.
- Although each instruction still takes 5 cycles (latency), the overall completion rate is improved (throughput).

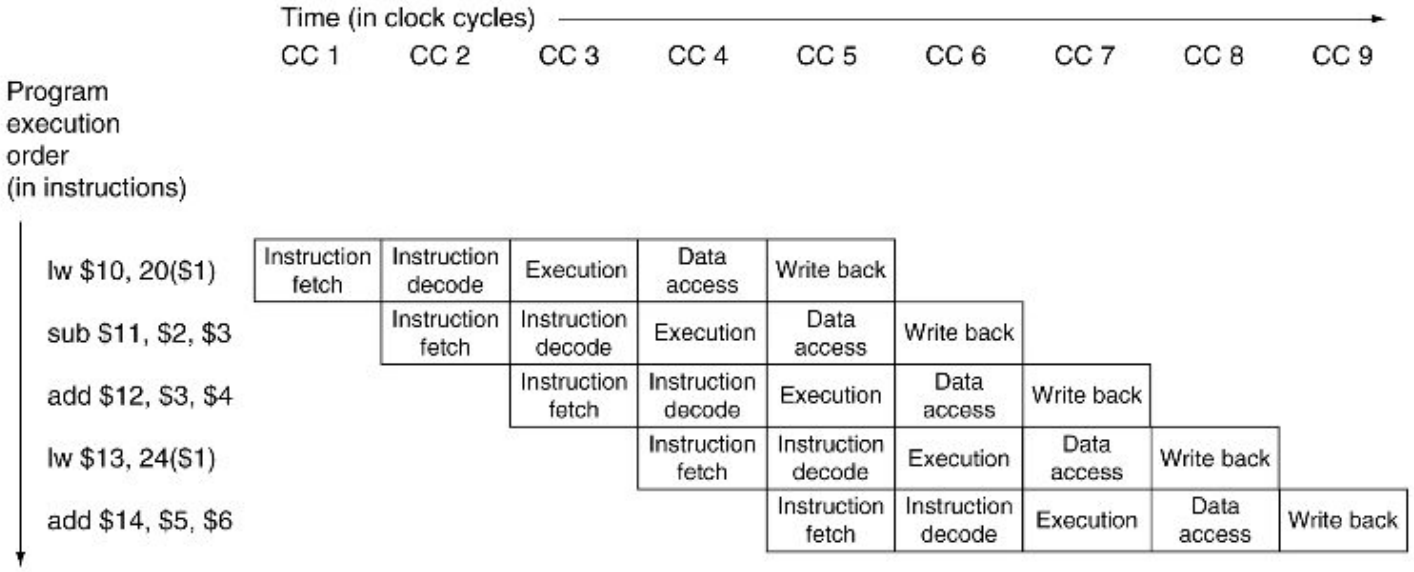
# Multi-Cycle Pipeline Diagram

- Form showing resource usage



# Multi-Cycle Pipeline Diagram

- Traditional form



# Multi-Cycle Pipeline Diagram

5-stage MIPS pipeline without stalls:

I1: lw $t0, 0($a0)

I2: add $t1, $t2, $t3

I3: sub $t4, $t5, $t6

I4: sw $t7, 4($a0)

At Cycle 5, which instruction occupies each pipeline stage?

A3.

| Stage | Instruction |
|---|---|
| IF | None |
| ID | I4 (sw) |
| EX | I3 (sub) |
| MEM | I2 (add) |
| WB | I1 (lw) |

# Pipeline Stage Occupancy

Time (in clock cycles) →

| Instruction | CC 1 | CC 2 | CC 3 | CC 4 | CC 5 | CC 6 | CC 7 |
|---|---|---|---|---|---|---|---|
| I1: lw $t1, 4($a0) | IF | ID | EX | MEM | WB | | |
| I2: sw $t2, 8($a0) | | IF | ID | EX | MEM | WB (no write) | |
| I3: add $t3, $t4, $t5 | | | IF | ID | EX | MEM | WB |

| Clock Cycle | Instruction in each pipeline stage during each clock cycle | | | | |
|---|---|---|---|---|---|
| | IF | ID | EX | MEM | WB |
| CC 1 | I1 | — | — | — | — |
| CC 2 | I2 | I1 | — | — | — |
| CC 3 | I3 | I2 | I1 | — | — |
| CC 4 | — | I3 | I2 | I1 | — |
| CC 5 | — | — | I3 | I2 | I1 |
| CC 6 | — | — | — | I3 | I2 (no write) |
| CC 7 | — | — | — | — | I3 |

# Pipeline Stage Occupancy

Time (in clock cycles) →

| Instruction | CC 1 | CC 2 | CC 3 | CC 4 | CC 5 | CC 6 | CC 7 |
|---|---|---|---|---|---|---|---|
| I1: lw $t1, 0($a0) | IF | ID | EX | MEM | WB | | |

n ideal 5-stage MIPS pipeline (I

MEM, WB) with no stalls.

Instruction I1 enters the IF stage at CC1, and each subsequent instruction enters IF one cycle later.

If I3 is in the MEM stage, determine the current clock cycle and the stage occupied by I5.

A4.

I3 enters IF at CC3.

I3 reaches MEM three cycles later, at CC6.

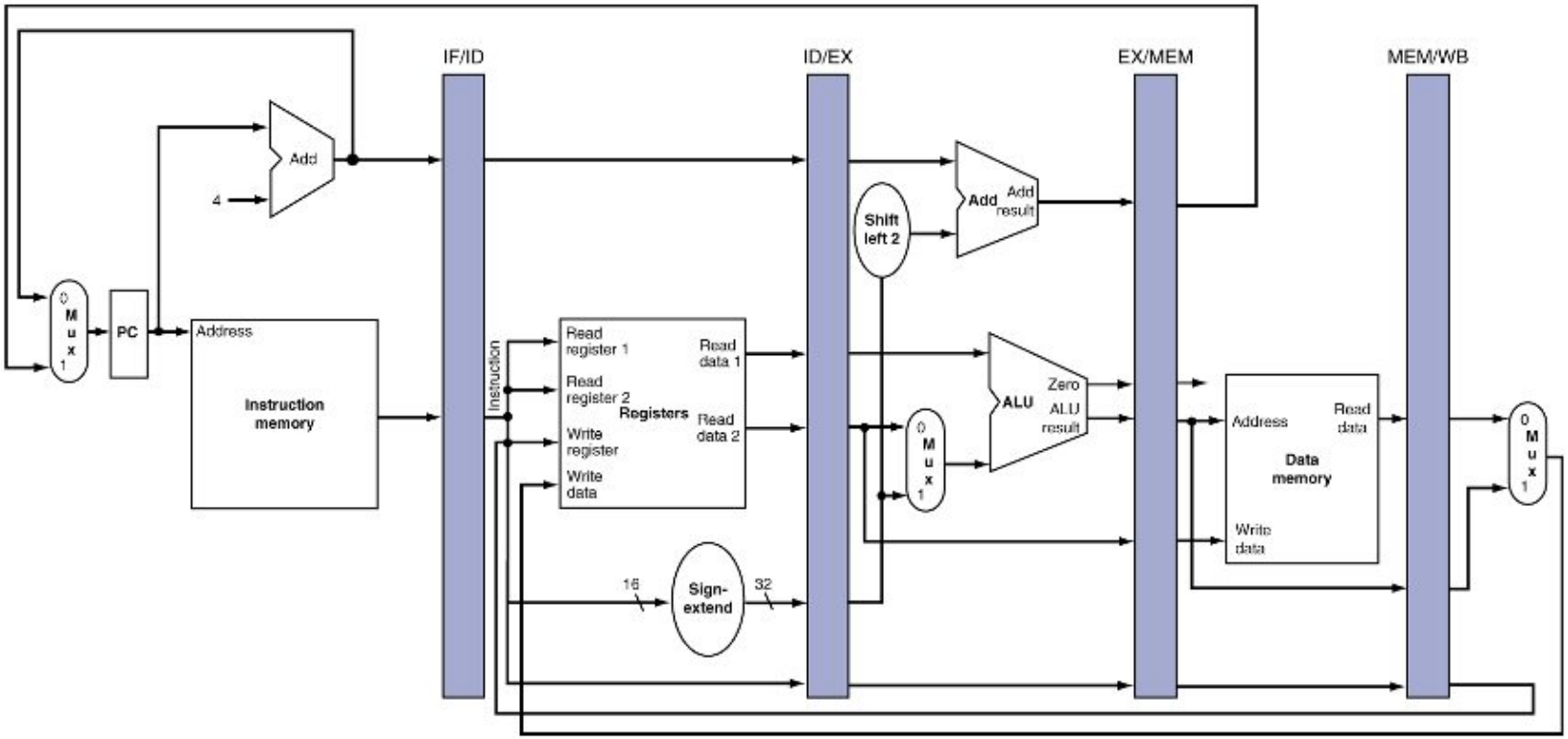
I5 enters IF at CC5 and reaches ID at CC6.

Therefore:

# Diagram

- State of pipeline in a given cycle

| add $14, $5, $6 | lw $13, 24 ($1) | add $12, $3, $4 | sub $11, $2, $3 | lw $10, 20($1) |
|---|---|---|---|---|
| Instruction fetch | Instruction decode | Execution | Memory | Write-back |



# Diagram

MIPS pipeline without stalls:

I1: lw $t0, 0($a0)

I2: sub $t1, $t2, $t3

I3: add $t4, $t5, $t6

I4: lw $t7, 8($a0)

I5: add $s0, $s1, $s2

At Clock Cycle 5 (CC5), identify the instruction using each hardware resource: Instruction Memory, Register File (Rea

A5.

| Hardware Resource | Instruction |
|---|---|
| Instruction Memory | I5 (add) |
| Register File (Read) | I4 (lw) |
| ALU | I3 (add) |
| Data Memory | I2 (sub) — no memory access |
| Register File (Write) | I1 (lw) |

# Summary

## 1 Load and Store Execution

- **lw** reads data from memory and writes it to a register.
- **sw** writes register data to memory without register write-back.

| | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| lw $t1, 4($a0) | Fetch instruction | Read registers | Compute address | Read data from memory | Write data to register |
| sw $t2, 8($a0) | Fetch instruction | Read registers | Compute address | Write data to memory | (no write) |

## 2 Instruction Overlapping

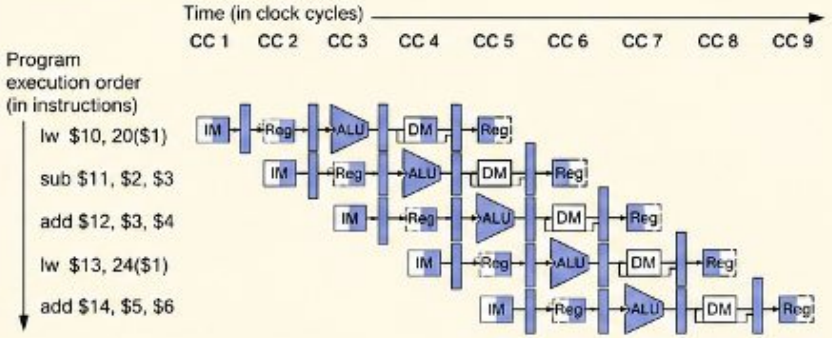
- Multiple instructions execute in different pipeline stages simultaneously.
- Ideally, a new instruction enters the pipeline every clock cycle after the pipeline is filled.

Time (in clock cycles) →

| | CC 1 | CC 2 | CC 3 | CC 4 | CC 5 | CC 6 | CC 8 | CC 7 |
|---|---|---|---|---|---|---|---|---|
| I1: lw $t1, 4($a0) | IF | ID | EX | MEM | WB | | | |
| I2: sw $t2, 8($a0) | | IF | ID | EX | MEM | WB (no write) | | |
| I3: add $t3, $t4, $t5 | | | IF | ID | EX | MEM | WB | |

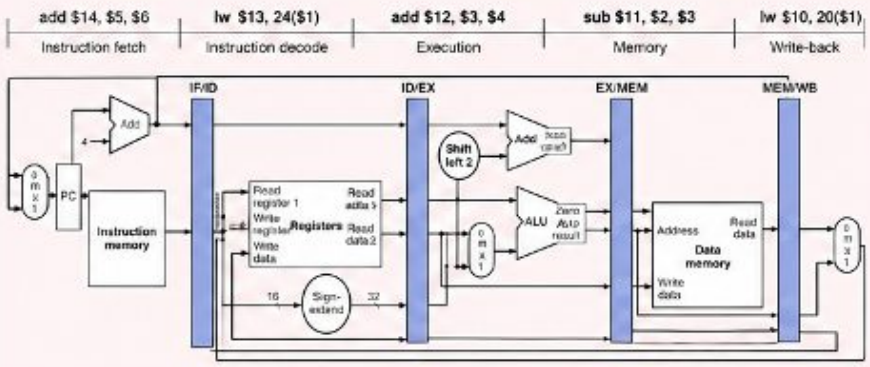
## 3 Multi-Cycle Pipeline Diagram

- Shows how each instruction progresses through pipeline stages across clock cycles.
- Helps us analyze instruction timing and hardware resource usage (e.g., IM, Reg, ALU, DM).




## 4 Single-Cycle Pipeline Diagram

- Shows which instruction occupies each pipeline stage in a specific clock cycle.
- Connects instruction timing to the actual pipeline datapath.




➜ **Pipelining improves instruction throughput by overlapping execution across multiple stages.**

# Section 4-3 Quick Questions

**Q1.** Given the EX, MEM, and WB operations, identify the MIPS instruction being executed.

**Q2.** For sw $t2, 12($a0), determine the ALU result, Read Data 2, and memory operation.

**Q3.** Given four instructions in a 5-stage pipeline, identify the instruction occupying each stage at CC5.

**Q4.** If I3 is in the MEM stage, determine the current clock cycle and the stage occupied by I5.

**Q5.** At CC5, identify which instruction uses each hardware resource: Instruction Memory, Register File (Read), ALU, Data Memory, and Register File (Write).

# Exercise 1 — Load and Store Datapath Tracing

**Consider the following MIPS instructions:**

I1: lw $t0, 8($a0)

I2: sw $t1, 12($a1)

(a) Calculate the effective address of each instruction.

(b) Determine the ALU result and store data in the EX/MEM register for I2.

(c) Determine the MEM operation

| Register / Memory | Value |
|---|---|
| $a0 | 0x10010000 |
| $a1 | 0x10020000 |
| $t1 | 0xABCDEF12 |
| Mem[0x10010008] | 0x12345678 |

## Answer

(a) Effective Addresses

- I1: 0x10010000 + 8 = 0x10010008
- I2: 0x10020000 + 12 = 0x1002000C

(b) EX/MEM Values for I2

- ALU result = 0x1002000C
- Store data = 0xABCDEF12

(c) MEM and WB Operations

- I1 (MEM): Read Mem[0x10010008] = 0x12345678
- I1 (WB): $t0 ← 0x12345678
- I2 (MEM): Mem[0x1002000C] ← 0xABCDEF12
- I2 (WB): No register write

**Key Point:** The EX stage calculates effective addresses, while MEM and WB stages perform instruction-specific operations.

# Exercise 2 — Multi-Cycle Pipeline Diagram

Consider the following instructions in an ideal 5-stage MIPS pipeline (IF, ID, EX, MEM, WB), without stalls.

I1: lw $t0, 0($a0)

I2: add $t1, $t2, $t3

I3: sub $t4, $t5, $t6

I4: lw $t7, 8($a0)

I5: add $s0, $s1, $s2

(a) How many instructions are active in the pipeline at CC4?

(b) At which clock cycle does I3 complete the WB stage?

(c) How many clock cycles are required to complete all five instructions?

**Answer**

(a) 4 instructions (I1–I4).

(b) CC7.

(c) 9 clock cycles.

**Key Point:** In an ideal 5-stage pipeline, completing N instructions requires N + 4 clock cycles.

# Exercise 3 — Single-Cycle Pipeline Analysis

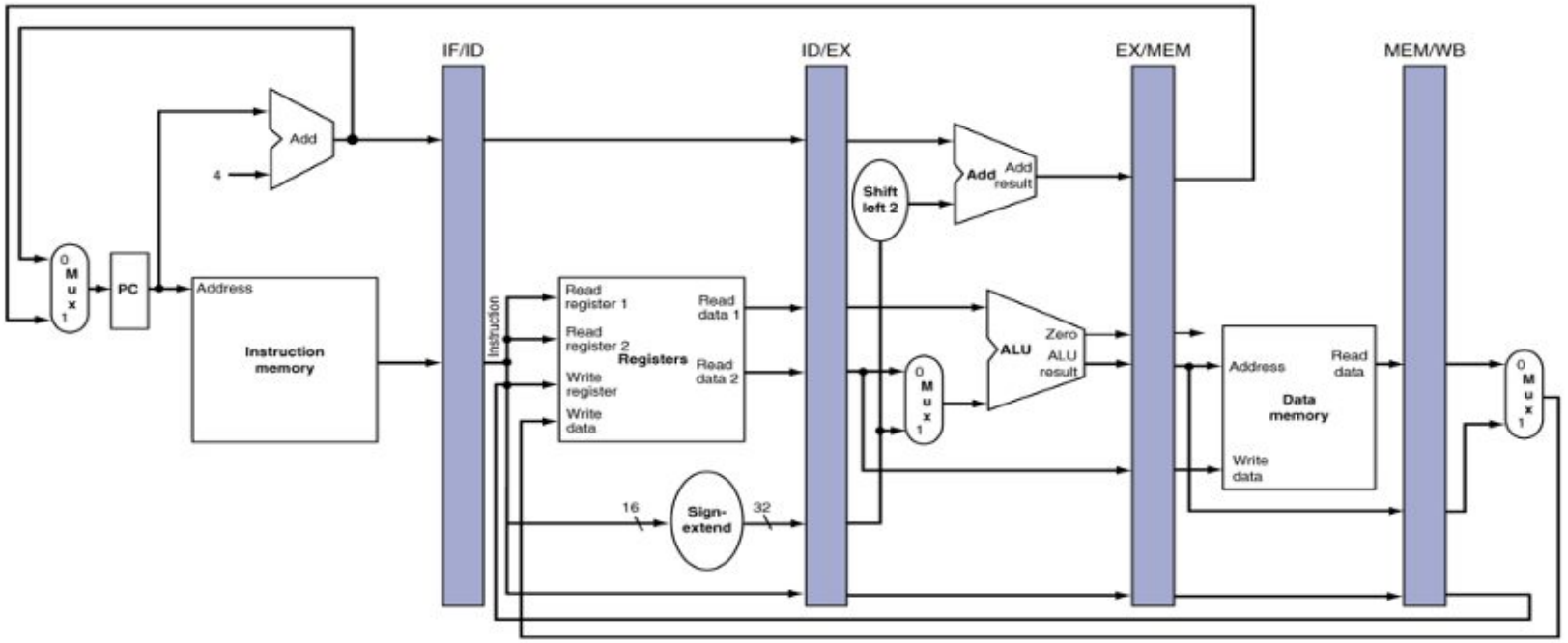
At CC3, use the datapath diagram to answer:

(a) Which instruction is associated with ID/EX, EX/MEM, and MEM/WB?

(b) Which instruction generates an ALU result? Where is it stored next?

(c) Which instruction performs WB? Identify the source and d



# Exercise 3 — Answer

## (a) Pipeline Register Contents at CC5

| | |
|---|---|
| **ID/EX** | I3: add $12, $3, $4 |
| **EX/MEM** | I2: sub $11, $2, $3 |
| **MEM/WB** | I1: lw $10, 20($1) |

## (b) ALU Result Transfer

I3 computes $3 + $4 in EX. The result is captured in EX/MEM at the end of CC5.

## (c) Register Write-Back

I1 (lw) writes memory-read data from MEM/WB into register $10.

**Key Point:** Different instructions occupy different stages and use distinct data paths.